ISEL
DEETC
Departamento de Engenharia Electrónica e de Telecomunicações e de Computadores

Aula 5

Sumário

- Exercícios de simplificação de funções com mapa de Karnaugh
- VHDL - descrição estrutural
- Introdução ao laboratório A

Exercícios

- Simplifique as seguintes funções com recurso a mapa de Karnaugh

$F_1 = \overline{A}.B + A.B$

$F_2 = \overline{A}.B + A.B + \overline{A}.\overline{B}$

$F_3 = \overline{A}.(\overline{A} + B)$

$F_4 = \overline{A}.(A + B)$

$F_5 = A\overline{B}C + ABC + AB\overline{C}$

$F_6 = (A + AB).(C + \overline{C}D) + \overline{A}D$

$F_7 = (A + B + C).(A + \overline{B} + C)$

$F_8(A, B, C, D) = \sum m(3,6,7,13,14,15)$

$F_9 = \overline{A}B + AC + BC$

$F_{10} = \overline{A}\,\overline{C} + \overline{A}D + \overline{A}B + A\overline{D} + \overline{B}\,\overline{D}$

$F_{11}(A, B, C, D) = \prod M(0, 1, 2, 4,5,8,9,10)$

$F_{12} = \overline{(A + \overline{B}).(C.\overline{D} + B)}$

$F_{13} = \overline{\overline{(A + B)}.(\overline{B} + C)}$

$F_{14} = \overline{\overline{A} + D}.\overline{(B + \overline{C})}.\overline{(\overline{C} + D)}$

Exercícios (soluções)

- Simplifique as seguintes funções com recurso a mapa de Karnaugh

$F_1 = \overline{A}.B + A.B = B$

$F_2 = \overline{A}.B + A.B + \overline{A}.\overline{B} = B + \overline{A}$

$F_3 = \overline{A}.(\overline{A} + B) = \overline{A}$

$F_4 = \overline{A}.(A + B) = \overline{A}.B$

$F_5 = A\overline{B}C + ABC + AB\overline{C} = AC + AB$

$F_6 = (A + AB).(C + \overline{C}D) + \overline{A}D = AC + D$

$F_7 = (A + B + C).(A + \overline{B} + C) = A + C$

$F_8(A, B, C, D) = \sum m(3,6,7,13,14,15) = BC + ABD + \overline{A}CD$

$F_9 = \overline{A}B + AC + BC = \overline{A}B + AC$

$F_{10} = \overline{A}\,\overline{C} + \overline{A}D + \overline{A}B + A\overline{D} + \overline{B}\,\overline{D} = \overline{A} + \overline{D}$

$F_{11}(A, B, C, D) = \prod M(0, 1, 2, 4,5,8,9,10) = BC + AB + CD$

$F_{12} = \overline{(A + \overline{B}).(C.\overline{D} + B)} = \overline{B}.\overline{C} + \overline{B}.D + \overline{A}.B$

$F_{13} = \overline{\overline{(A + B)}.(\overline{B} + C)} = A + \overline{B} + C + D$

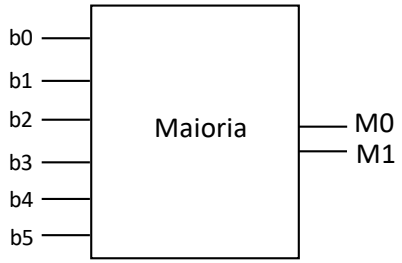
$F_{14} = \overline{\overline{A} + D}.\overline{(B + \overline{C})}.\overline{(\overline{C} + D)} = A\overline{B}C\overline{D}$

Hierarquia e reutilização de blocos funcionais

- No projeto de circuitos digitais complexos, a estratégia mais comum para desenhar a solução contempla o seguinte:
  - Dividir o problema em vários problemas mais pequenos (de resolução mais fácil)
  - Reutilizar blocos funcionais (módulos) previamente projetados:
    - Cada bloco funcional já está devidamente caracterizado e validado
    - Permite a redução do tempo necessário para o projeto
    - Reduz, potencialmente, os custos de produção
- O projeto modular utiliza vários módulos organizados hierarquicamente para construir o circuito lógico completo

Projeto hierárquico (exemplo)

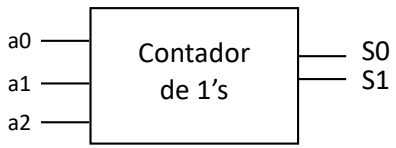
- Queremos implementar um circuito combinatório que indica se existem mais bits a 1 ou a 0 numa palavra de 6 bits. O resultado, codificado numa saída de 2 bits, é o seguinte:
  - 01 se #1's > #0's
  - 11 se #1's < #0's
  - 00 se #1's = #0's
  - E.g. – para a entrada "110011", a saída deverá ser "01"

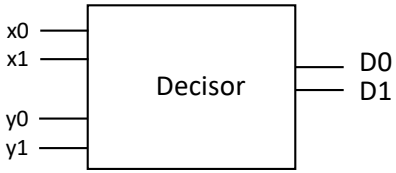


  - Duas funções com 6 entradas
    - Tabela de verdade com 6 entradas (64 possíveis combinações)

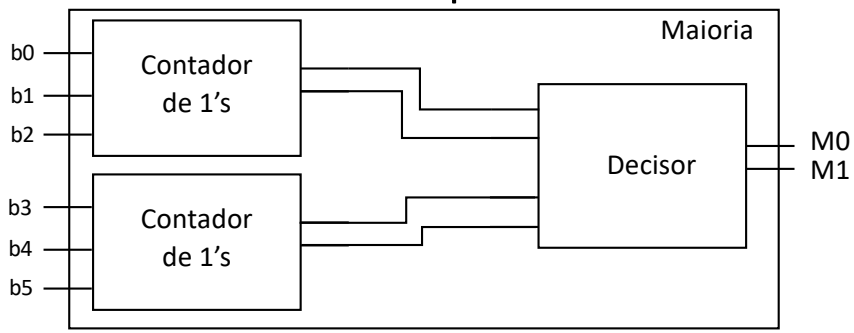
Projeto hierárquico (exemplo)

- Reduzir a complexidade (dividir o problema)
  - Utilizar blocos funcionais de menor complexidade





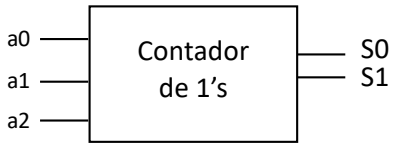
  - Desenhar circuito complexo à custa dos módulos mais simples



Projeto hierárquico (exemplo)

- Contador de 1's

a2	a1	a0	S1	S0
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



Projeto hierárquico (exemplo)

- Contador de 1's

a2	a1	a0	S1	S0
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

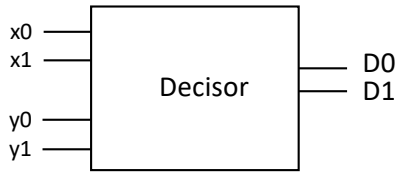
$$S0 = a2 \oplus a1 \oplus a0$$

$$S1 = a2\ a1 + a1\ a0 + a2\ a0$$

Projeto hierárquico (exemplo)

- Decisor

y1	y0	x1	x0	D1	D0
0	0	0	0	1	1
0	0	0	1	1	1
0	0	1	0	1	1
0	0	1	1	0	0
0	1	0	0	1	1
0	1	0	1	1	1
0	1	1	0	0	0
0	1	1	1	0	1
1	0	0	0	1	1
1	0	0	1	0	0
1	0	1	0	0	1
1	0	1	1	0	1
1	1	0	0	0	0
1	1	0	1	0	1
1	1	1	0	0	1
1	1	1	1	0	1



01 se #1's > #0's
11 se #1's < #0's
00 se #1's = #0's

Projeto hierárquico (exemplo)

- Decisor

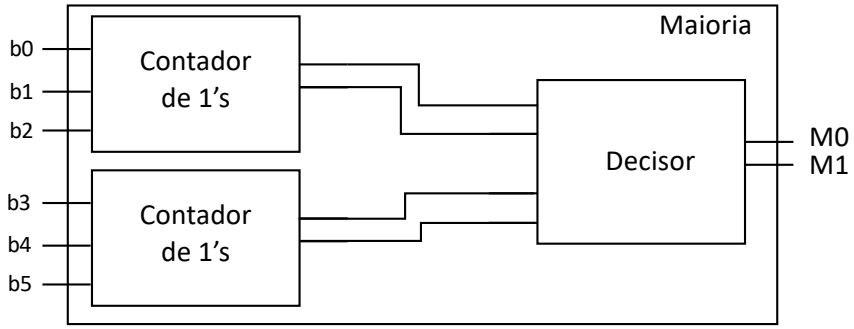
y1	y0	x1	x0	D1	D0
0	0	0	0	1	1
0	0	0	1	1	1
0	0	1	0	1	1
0	0	1	1	0	0
0	1	0	0	1	1
0	1	0	1	1	1
0	1	1	0	0	0
0	1	1	1	0	1
1	0	0	0	1	1
1	0	0	1	0	0
1	0	1	0	0	1
1	0	1	1	0	1
1	1	0	0	0	0
1	1	0	1	0	1
1	1	1	0	0	1
1	1	1	1	0	1

$$D0 = \overline{y1}\,\overline{x1} + y1x1 + y0x0 + \overline{y0}\,\overline{x0}$$

$$D1 = \overline{y1}\,\overline{x1} + \overline{y1}\,\overline{y0}\,\overline{x0} + \overline{y0}\,\overline{x0}\,\overline{x1}$$

Projeto hierárquico (exemplo)

- Uma vez tendo definido e implementado os módulos funcionais mais simples, é possível implementar o circuito mais complexo, fazendo as devidas interligações

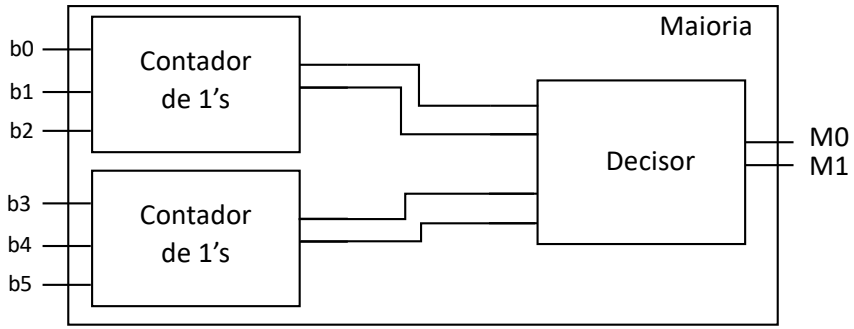


Projeto hierárquico (exemplo) VHDL – descrição estrutural

- Numa descrição do tipo *Behavioral* (comportamental) usam-se primitivas que descrevem o funcionamento da *entity* (entidade)
  - E.g. descrevem-se as funções lógicas de cada uma das saídas
- Numa descrição do tipo *Structural* (estrutural) usam-se vários componentes e as respetivas interligações para implementar a funcão desejada
  - É o tipo de descrição usada quando temos um bloco funcional que é construído à custa de módulos mais simples (projeto hierárquico)

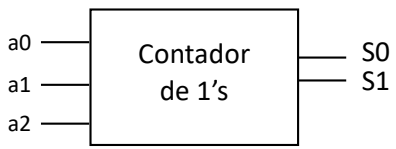
Projeto hierárquico (exemplo) VHDL – descrição estrutural

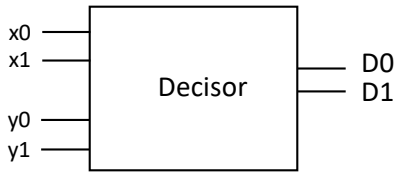
- Diagrama de blocos do circuito a implementar:



Projeto hierárquico (exemplo) VHDL – descrição estrutural

- Funções lógicas de cada um dos módulos funcionais mais simples:





$$S0 = a2 \oplus a1 \oplus a0$$

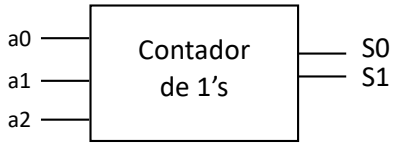
$$S1 = a2\,a1 + a1\,a0 + a2\,a0$$

$$D0 = \overline{y1}\,\overline{x1} + y1x1 + y0x0 + \overline{y0}\,\overline{x0}$$

$$D1 = \overline{y1}\,\overline{x1} + \overline{y1}\,\overline{y0}\,\overline{x0} + \overline{y0}\,\overline{x0}\,\overline{x1}$$

Projeto hierárquico (exemplo) VHDL – descrição estrutural

- Para cada um dos módulos funcionais, fazer a sua descrição em VHDL (comportamental) – uma entidade para cada módulo
  - Se já existir uma descrição prévia do módulo, reutilizá-la



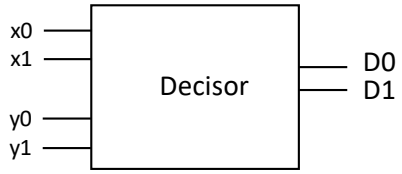
$$S0 = a2 \oplus a1 \oplus a0$$

$$S1 = a2\ a1 + a1\ a0 + a2\ a0$$

```
library ieee;
use ieee.std_logic_1164.all;

entity oneCounter is
   port (
      a : in std_logic_vector(2 downto 0);
      S : out std_logic_vector(1 downto 0)
   );
end oneCounter;


architecture behavioral of oneCounter is
begin

   S(0) <= a(2) xor a(1) xor a(0);
   S(1) <= (a(2) and a(1)) or (a(1) and a(0)) or (a(2) and a(0));

end behavioral;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

- Para cada um dos módulos funcionais, fazer a sua descrição em VHDL (comportamental) – uma entidade para cada módulo
  - Se já existir uma descrição prévia do módulo, reutilizá-la



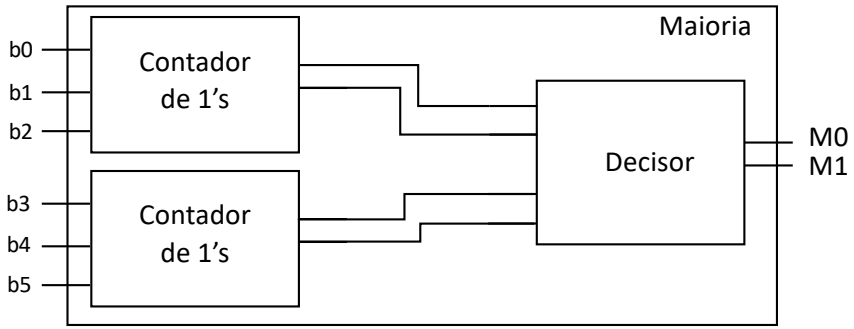
$$D0 = \overline{y1}\,\overline{x1} + y1x1 + y0x0 + \overline{y0}\,\overline{x0}$$

$$D1 = \overline{y1}\,\overline{x1} + \overline{y1}\,\overline{y0}\,\overline{x0} + \overline{y0}\,\overline{x0}\,\overline{x1}$$

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity decisionLogic is
   port (
      x : in std_logic_vector(1 downto 0);
      y : in std_logic_vector(1 downto 0);
      D : out std_logic_vector(1 downto 0)
   );
end decisionLogic;

architecture behavioral of decisionLogic is

   signal ny1nx1, ny0nx0 : std_logic;

begin

   ny1nx1 <= not y(1) and not x(1);
   ny0nx0 <= not y(0) and not x(0);

   D(0) <= ny1nx1 or (y(1) and x(1)) or (y(0) and x(0)) or ny0nx0;
   D(1) <= ny1nx1 or (not y(1) and ny0nx0) or (ny0nx0 and not x(1));

end behavioral;
```

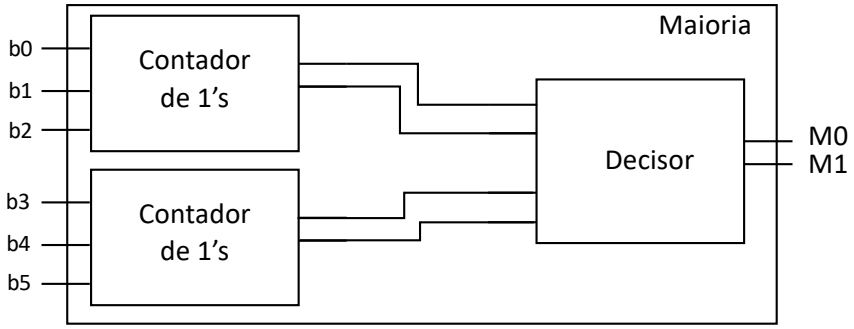
Projeto hierárquico (exemplo) VHDL – descrição estrutural

- O circuito “Maioria” é construído à custa de ligações entre vários módulos:
  - 2x Contador de 1’s
  - 1x Decisor



Projeto hierárquico (exemplo) VHDL – descrição estrutural

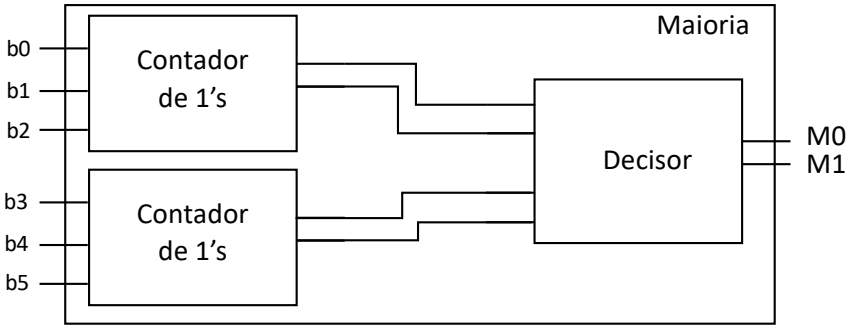
- Definir as entradas e saídas da entidade maioria (*majority*)




```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity majority is
   port (
      b : in std_logic_vector(5 downto 0);
      M : out std_logic_vector(1 downto 0)
   );
end majority;


architecture structural of majority is


begin


end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

- Declarar os componentes que vão ser usados na descrição da arquitetura




```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity majority is
   port (
      b : in std_logic_vector(5 downto 0);
      M : out std_logic_vector(1 downto 0)
   );
end majority;


architecture structural of majority is

   component oneCounter
      port (
         a : in std_logic_vector(2 downto 0);
         S : out std_logic_vector(1 downto 0)
      );
   end component;

   component decisionLogic
      port (
         x : in std_logic_vector(1 downto 0);
         y : in std_logic_vector(1 downto 0);
         D : out std_logic_vector(1 downto 0)
      );
   end component;

begin


end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

- A primitiva “component” é utilizada para declarar os componentes que serão utilizados na descrição de uma arquitetura (descrição estrutural)

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity oneCounter is
   port (
      a : in std_logic_vector(2 downto 0);
      S : out std_logic_vector(1 downto 0)
   );
end oneCounter;


architecture behavioral of oneCounter is
begin

   S(0) <= a(2) xor a(1) xor a(0);
   S(1) <= (a(2) and a(1)) or (a(1) and a(0)) or (a(2) and a(0));

end behavioral;
```

O nome e os portos têm de ser iguais

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity majority is
   port (
      b : in std_logic_vector(5 downto 0);
      M : out std_logic_vector(1 downto 0)
   );
end majority;


architecture structural of majority is

   component oneCounter
      port (
         a : in std_logic_vector(2 downto 0);
         S : out std_logic_vector(1 downto 0)
      );
   end component;

   component decisionLogic
      port (
         x : in std_logic_vector(1 downto 0);
         y : in std_logic_vector(1 downto 0);
         D : out std_logic_vector(1 downto 0)
      );
   end component;

begin


end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

- A primitiva “component” é utilizada para declarar os componentes que serão utilizados na descrição de uma arquitetura (descrição estrutural)

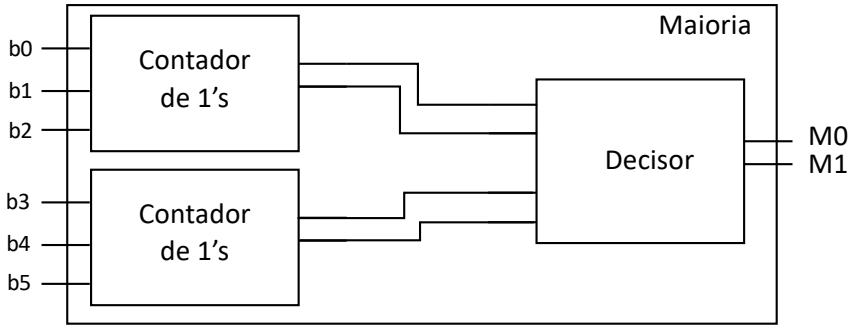
```
library ieee;
use ieee.std_logic_1164.all;

entity decisionLogic is
   port (
      x : in std_logic_vector(1 downto 0);
      y : in std_logic_vector(1 downto 0);
      D : out std_logic_vector(1 downto 0)
   );
end decisionLogic;

architecture behavioral of decisionLogic is

   signal ny1nx1, ny0nx0 : std_logic;

begin

   ny1nx1 <= not y(1) and not x(1);
   ny0nx0 <= not y(0) and not x(0);

   D(0) <= ny1nx1 or (y(1) and x(1)) or (y(0) and x(0)) or ny0nx0;
   D(1) <= ny1nx1 or (not y(1) and ny0nx0) or (ny0nx0 and not x(1));

end behavioral;
```

O nome e os portos têm de ser iguais

```
library ieee;
use ieee.std_logic_1164.all;

entity majority is
   port (
      b : in std_logic_vector(5 downto 0);
      M : out std_logic_vector(1 downto 0)
   );
end majority;


architecture structural of majority is

   component oneCounter
      port (
         a : in std_logic_vector(2 downto 0);
         S : out std_logic_vector(1 downto 0)
      );
   end component;

   component decisionLogic
      port (
         x : in std_logic_vector(1 downto 0);
         y : in std_logic_vector(1 downto 0);
         D : out std_logic_vector(1 downto 0)
      );
   end component;

begin


end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

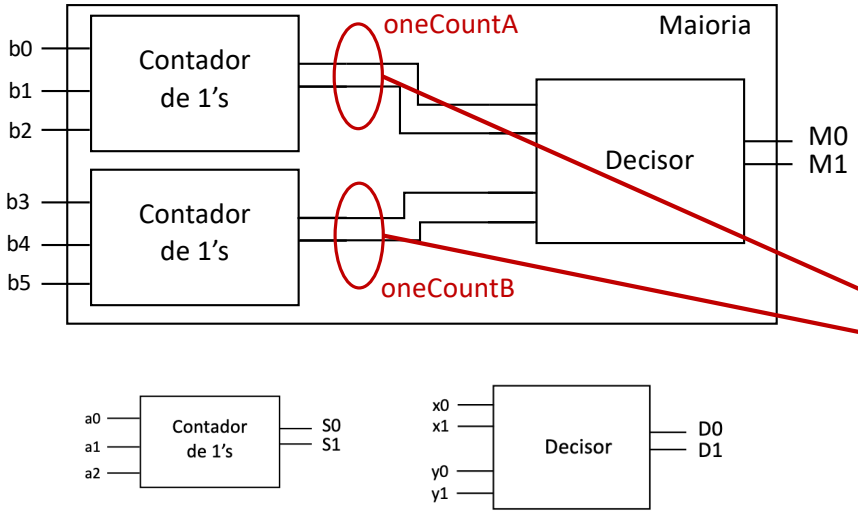
- Definir os sinais a serem utilizados




```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity majority is
   port (
      b : in std_logic_vector(5 downto 0);
      M : out std_logic_vector(1 downto 0)
   );
end majority;


architecture structural of majority is

   component oneCounter
      port (
         a : in std_logic_vector(2 downto 0);
         S : out std_logic_vector(1 downto 0)
      );
   end component;

   component decisionLogic
      port (
         x : in std_logic_vector(1 downto 0);
         y : in std_logic_vector(1 downto 0);
         D : out std_logic_vector(1 downto 0)
      );
   end component;

   signal oneCountA, oneCountB : std_logic_vector (1 downto 0);

begin


end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

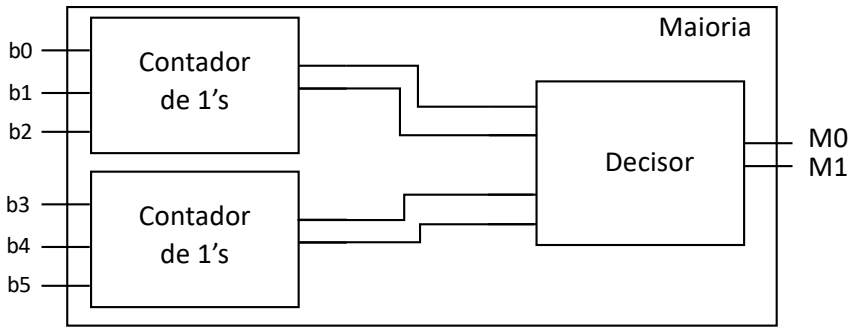
- Definir os sinais a serem utilizados




```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity majority is
   port (
      b : in std_logic_vector(5 downto 0);
      M : out std_logic_vector(1 downto 0)
   );
end majority;


architecture structural of majority is

   component oneCounter
      port (
         a : in std_logic_vector(2 downto 0);
         S : out std_logic_vector(1 downto 0)
      );
   end component;

   component decisionLogic
      port (
         x : in std_logic_vector(1 downto 0);
         y : in std_logic_vector(1 downto 0);
         D : out std_logic_vector(1 downto 0)
      );
   end component;

   signal oneCountA, oneCountB : std_logic_vector (1 downto 0);

begin


end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

- Instanciar os componentes que forem necessários




```
begin

   oneCountUnitA : oneCounter port map (
      a => ,
      S =>
   );

   oneCountUnitB : oneCounter port map (
      a => ,
      S =>
   );

   decisionUnit : decisionLogic port map (
      x => ,
      y => ,
      D =>
   );

end structural;
```

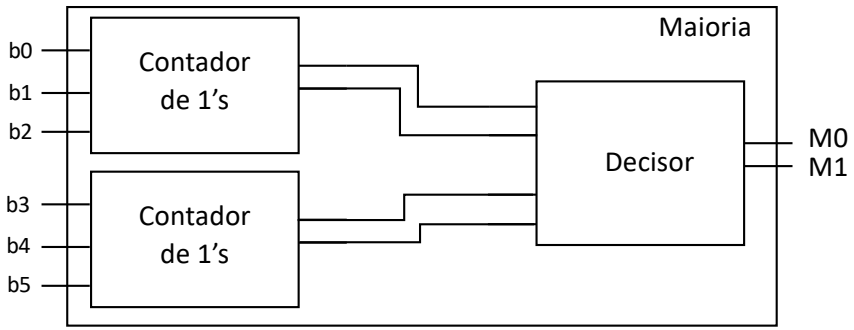
Projeto hierárquico (exemplo) VHDL – descrição estrutural

- Instanciar os componentes que forem necessários

Nome da instância
(tem de ser único)

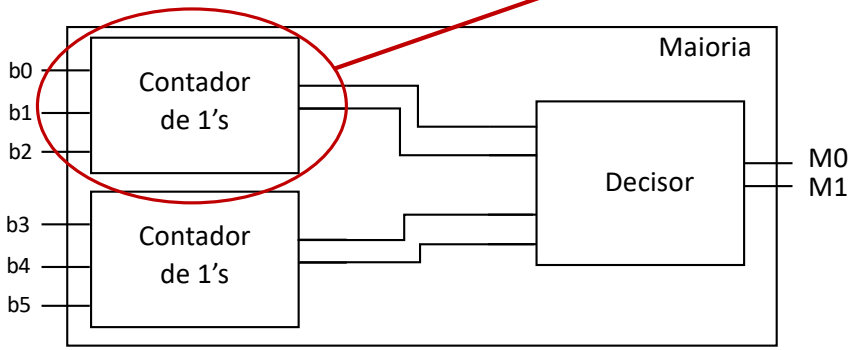
Componente a instanciar

```
begin

   oneCountUnitA : oneCounter port map (
      a => ,
      S =>
   );

   oneCountUnitB : oneCounter port map (
      a => ,
      S =>
   );

   decisionUnit : decisionLogic port map (
      x => ,
      y => ,
      D =>
   );

end structural;
```



Projeto hierárquico (exemplo) VHDL – descrição estrutural

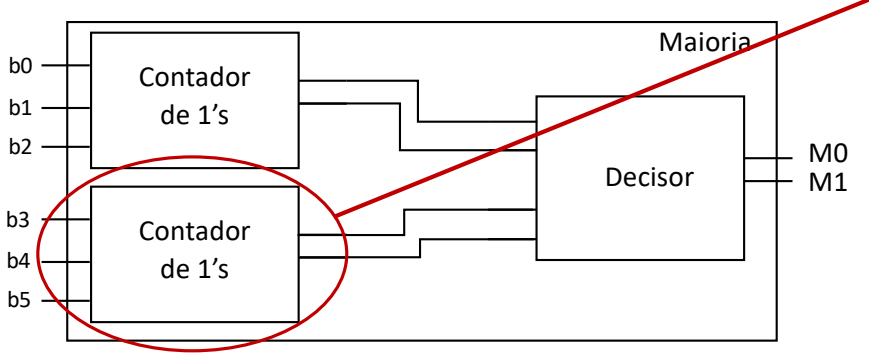
- Instanciar os componentes que forem necessários




```vhdl
begin

   oneCountUnitA : oneCounter port map (
      a => ,
      S =>
   );

   oneCountUnitB : oneCounter port map (
      a => ,
      S =>
   );

   decisionUnit : decisionLogic port map (
      x => ,
      y => ,
      D =>
   );

end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

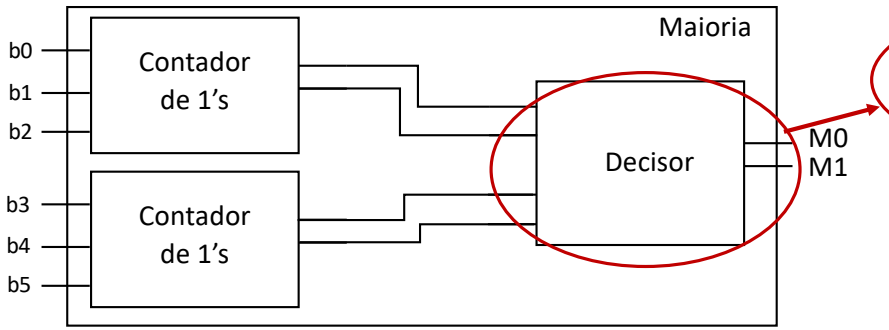
- Instanciar os componentes que forem necessários




```
begin

   oneCountUnitA : oneCounter port map (
      a => ,
      S =>
   );

   oneCountUnitB : oneCounter port map (
      a => ,
      S =>
   );

   decisionUnit : decisionLogic port map (
      x => ,
      y => ,
      D =>
   );

end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

- Instanciar os componentes que forem necessários




```
begin

   oneCountUnitA : oneCounter port map (
      a => ,
      S =>
   );

   oneCountUnitB : oneCounter port map (
      a => ,
      S =>
   );

   decisionUnit : decisionLogic port map (
      x => ,
      y => ,
      D =>
   );

end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

- Instanciar os componentes que forem necessários
- É necessário mapear (ligar) os portos dos componentes a entradas ou saídas ou sinais da entidade (*port map*)

```
begin

   oneCountUnitA : oneCounter port map (
      a => ???,
      S => ???
   );

   oneCountUnitB : oneCounter port map (
      a => ???,
      S => ???
   );

   decisionUnit : decisionLogic port map (
      x => ???,
      y => ???,
      D => ???
   );

end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

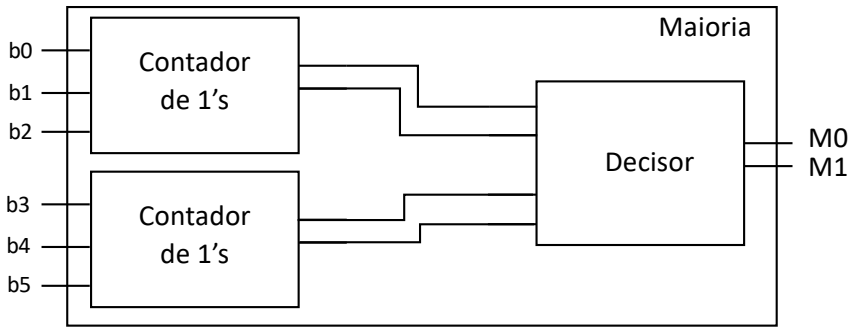
- Instanciar os componentes que forem necessários
- É necessário mapear (ligar) os portos dos componentes a entradas ou saídas ou sinais da entidade (*port map*)

```
component oneCounter
   port (
      a : in std_logic_vector(2 downto 0);
      S : out std_logic_vector(1 downto 0)
   );
end component;

component decisionLogic
   port (
      x : in std_logic_vector(1 downto 0);
      y : in std_logic_vector(1 downto 0);
      D : out std_logic_vector(1 downto 0)
   );
end component;
```

```
begin

   oneCountUnitA : oneCounter port map (
      a => ???,
      S => ???
   );

   oneCountUnitB : oneCounter port map (
      a => ???,
      S => ???
   );

   decisionUnit : decisionLogic port map (
      x => ???,
      y => ???,
      D => ???
   );

end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

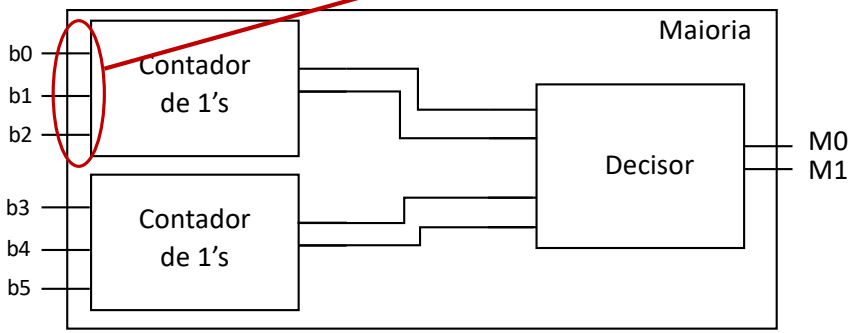
- Descrever a arquitetura à custa das ligações entre entradas e saídas da entidade e dos componentes internos




```vhdl
begin

   oneCountUnitA : oneCounter port map (
      a => b(2 downto 0),
      S => oneCountA
   );

   oneCountUnitB : oneCounter port map (
      a => b(5 downto 3),
      S => oneCountB
   );

   decisionUnit : decisionLogic port map (
      x => oneCountA,
      y => oneCountB,
      D => M
   );

end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

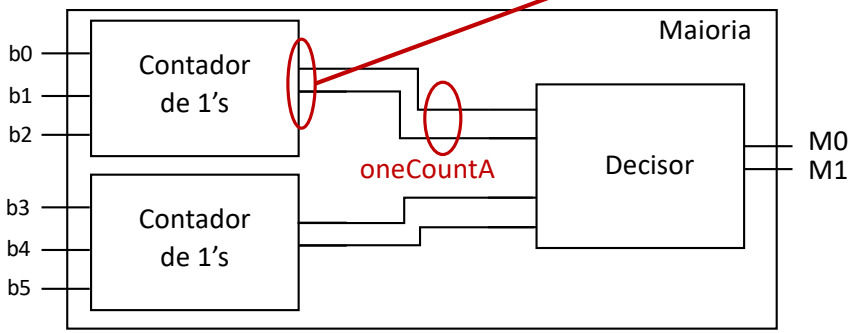
- Descrever a arquitetura à custa das ligações entre entradas e saídas da entidade e dos componentes internos




```vhdl
begin

   oneCountUnitA : oneCounter port map (
      a => b(2 downto 0),
      S => oneCountA
   );

   oneCountUnitB : oneCounter port map (
      a => b(5 downto 3),
      S => oneCountB
   );

   decisionUnit : decisionLogic port map (
      x => oneCountA,
      y => oneCountB,
      D => M
   );

end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

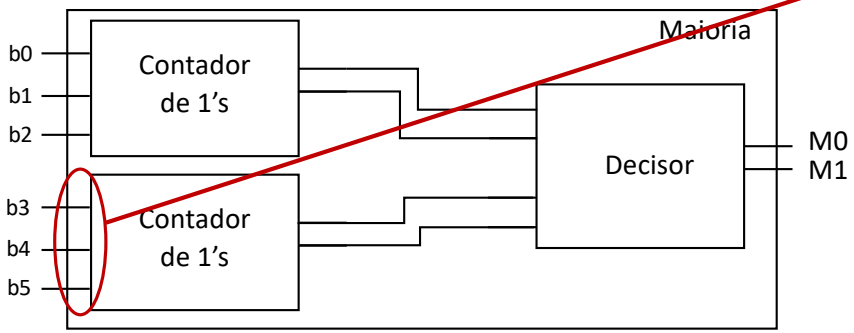
- Descrever a arquitetura à custa das ligações entre entradas e saídas da entidade e dos componentes internos




```vhdl
begin

   oneCountUnitA : oneCounter port map (
      a => b(2 downto 0),
      S => oneCountA
   );

   oneCountUnitB : oneCounter port map (
      a => b(5 downto 3),
      S => oneCountB
   );

   decisionUnit : decisionLogic port map (
      x => oneCountA,
      y => oneCountB,
      D => M
   );

end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

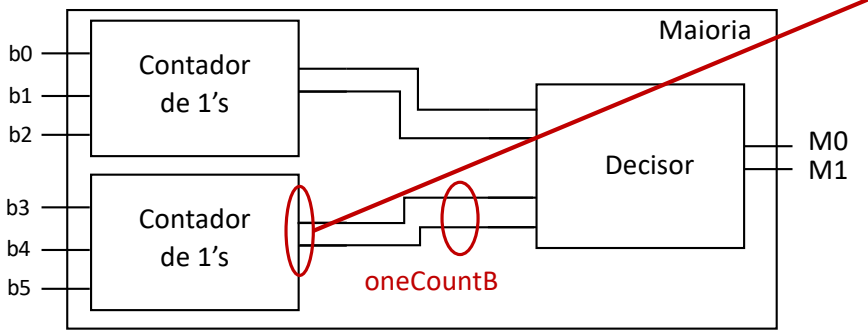
- Descrever a arquitetura à custa das ligações entre entradas e saídas da entidade e dos componentes internos




```
begin

   oneCountUnitA : oneCounter port map (
      a => b(2 downto 0),
      S => oneCountA
   );

   oneCountUnitB : oneCounter port map (
      a => b(5 downto 3),
      S => oneCountB
   );

   decisionUnit : decisionLogic port map (
      x => oneCountA,
      y => oneCountB,
      D => M
   );

end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

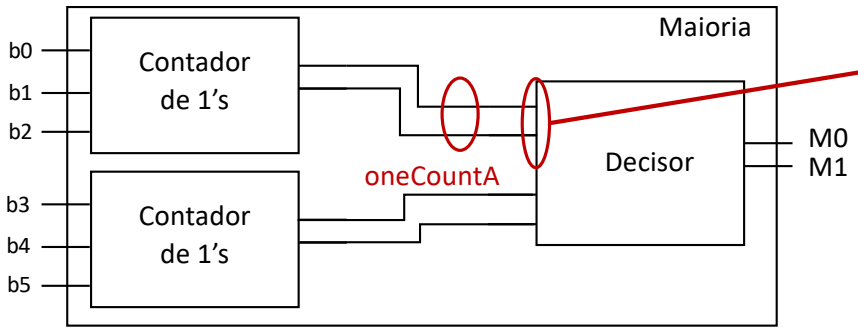
- Descrever a arquitetura à custa das ligações entre entradas e saídas da entidade e dos componentes internos




```vhdl
begin

   oneCountUnitA : oneCounter port map (
      a => b(2 downto 0),
      S => oneCountA
   );

   oneCountUnitB : oneCounter port map (
      a => b(5 downto 3),
      S => oneCountB
   );

   decisionUnit : decisionLogic port map (
      x => oneCountA,
      y => oneCountB,
      D => M
   );

end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

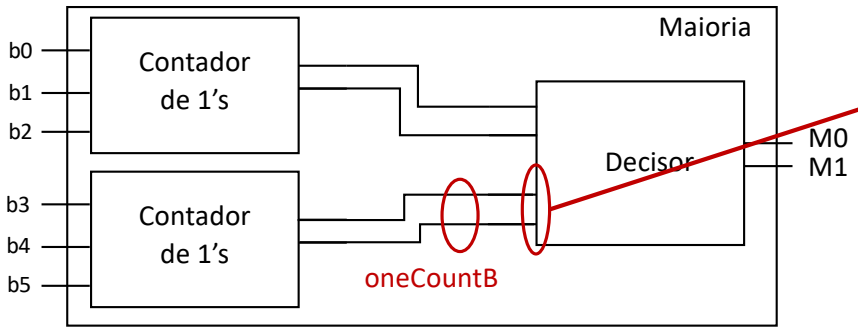
- Descrever a arquitetura à custa das ligações entre entradas e saídas da entidade e dos componentes internos




```
begin

   oneCountUnitA : oneCounter port map (
      a => b(2 downto 0),
      S => oneCountA
   );

   oneCountUnitB : oneCounter port map (
      a => b(5 downto 3),
      S => oneCountB
   );

   decisionUnit : decisionLogic port map (
      x => oneCountA,
      y => oneCountB,
      D => M
   );

end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

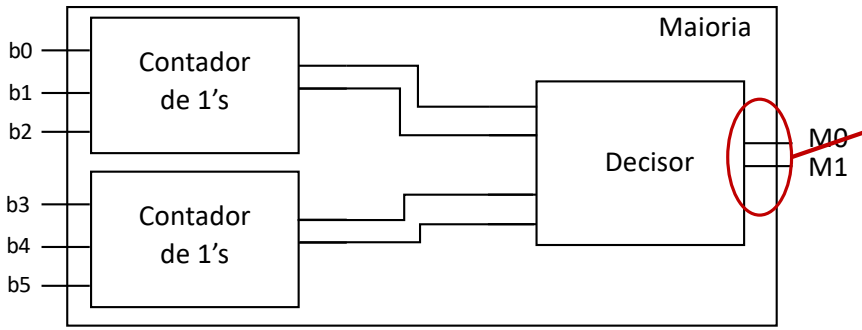
- Descrever a arquitetura à custa das ligações entre entradas e saídas da entidade e dos componentes internos




```vhdl
begin

   oneCountUnitA : oneCounter port map (
      a => b(2 downto 0),
      S => oneCountA
   );

   oneCountUnitB : oneCounter port map (
      a => b(5 downto 3),
      S => oneCountB
   );

   decisionUnit : decisionLogic port map (
      x => oneCountA,
      y => oneCountB,
      D => M
   );

end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

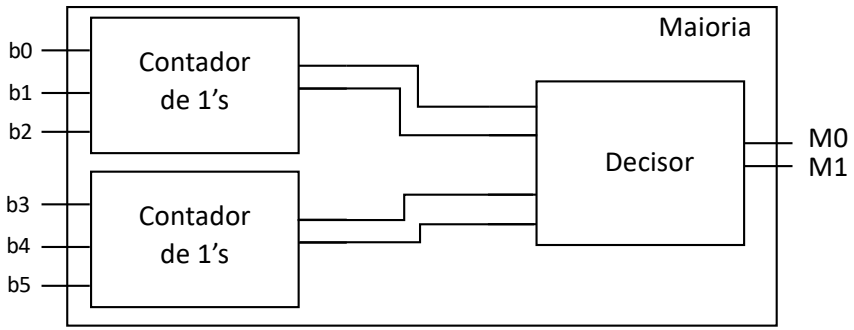
- Descrever a arquitetura à custa das ligações entre entradas e saídas da entidade e dos componentes internos

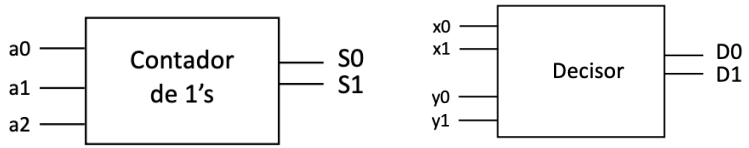



```vhdl
begin

   oneCountUnitA : oneCounter port map (
      a => b(2 downto 0),
      S => oneCountA
   );

   oneCountUnitB : oneCounter port map (
      a => b(5 downto 3),
      S => oneCountB
   );

   decisionUnit : decisionLogic port map (
      x => oneCountA,
      y => oneCountB,
      D => M
   );

end structural;
```

Projeto hierárquico (exemplo) VHDL – descrição estrutural

- Descrever a arquitetura à custa das ligações entre entradas e saídas da entidade e dos componentes internos






```
architecture structural of majority is

   component oneCounter
      port (
         a : in std_logic_vector(2 downto 0);
         S : out std_logic_vector(1 downto 0)
      );
   end component;

   component decisionLogic
      port (
         x : in std_logic_vector(1 downto 0);
         y : in std_logic_vector(1 downto 0);
         D : out std_logic_vector(1 downto 0)
      );
   end component;

   signal oneCountA, oneCountB : std_logic_vector (1 downto 0);

begin

   oneCountUnitA : oneCounter port map (
      a => b(2 downto 0),
      S => oneCountA
   );

   oneCountUnitB : oneCounter port map (
      a => b(5 downto 3),
      S => oneCountB
   );

   decisionUnit : decisionLogic port map (
      x => oneCountA,
      y => oneCountB,
      D => M
   );

end structural;
```

ISEL
DEETC
Departamento de Engenharia Electrónica e de Telecomunicações e de Computadores

Laboratório A – Parte I

Funções Lógicas – Tecnologia *TTL*

Implementação de funções com circuitos digitais

- Os circuitos digitais utilizam grandezas físicas (e.g. tensão, corrente) para representar (codificar) os valores lógicos binários da álgebra de Boole
  - Cada tecnologia tem o seu próprio mapeamento

- No caso da família lógica 74LSxxx, os valores lógicos binários são representados em dois intervalos de tensão
  - Valor de tensão HIGH – [2; 5] V
  - Valor de tensão LOW – [0; 0,8] V

- Adicionalmente, é necessário mapear os valores lógicos numéricos 1 e 0 nos possíveis valores da respetiva grandeza física
  - No caso da família lógica 74LSxxx, o valor lógico 1 pode ser mapeado no:
    - Valor de tensão HIGH – caso em que se denomina a entrada/saída como *active_high*
    - Valor de tensão LOW – caso em que se denomina a entrada/saída como *active_low*
  - Um mesmo circuito pode ter algumas das suas entradas/saídas *active_high* e outras *active_low*
    - É necessário verificar as características do respetivo circuito

Implementação de funções com circuitos digitais

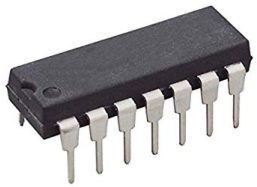
- Resumo para o trabalho de laboratório 1:
  - vamos apenas utilizar circuitos digitais com entradas e saídas *active_high*

Valor lógico (número)	Valor lógico (tensão)	Valor de tensão
1	HIGH	[2; 5] V
0	LOW	[0; 0,8] V

Mapeamento entre a representação numérica do valor lógico e o correspondente valor de tensão nos circuitos digitais 74LS00, 74LS02 e 74LS04 (entradas e saídas *active_high*)

Circuitos integrados

- Por norma, usar sempre circuitos integrados da mesma família (e.g. 74LSxxx); apenas usar de famílias diferentes se forem compatíveis!
- Referências dos circuitos a usar nesta aula de laboratório



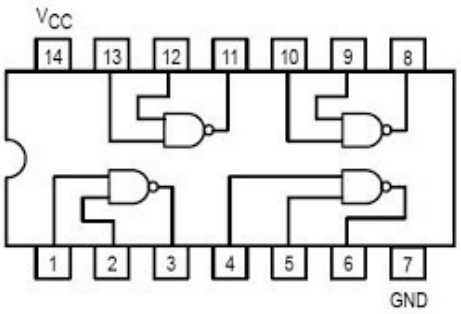
Referência	Função
74LS00	Quad 2-input NAND
74LS02	Quad 2-input NOR
74LS04	Hex Inverter

- Datasheets disponíveis online (e.g. https://www.futurlec.com/IC74LS00Series.shtml)

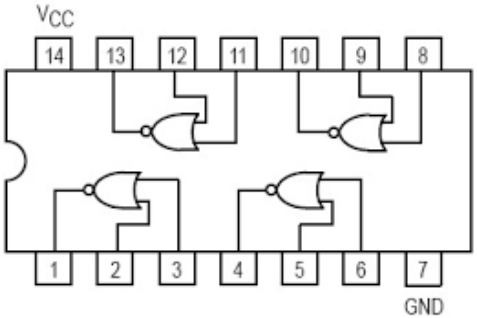
Circuitos integrados
Pinos

- Pinout

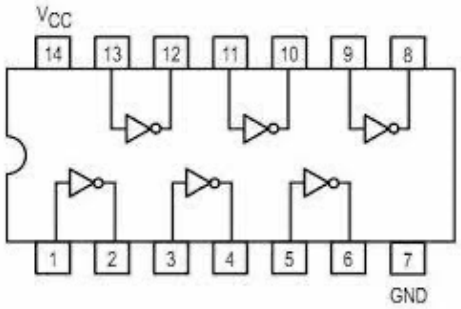
74LS00
Quad 2-input NAND



74LS02
Quad 2-input NOR



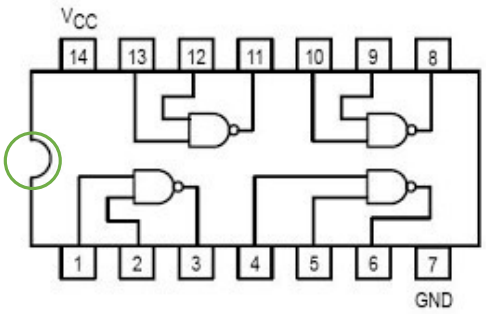
74LS04
Hex Inverter



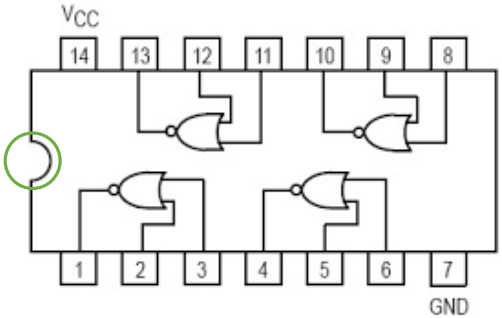
Circuitos integrados
Pinos

- Pinout

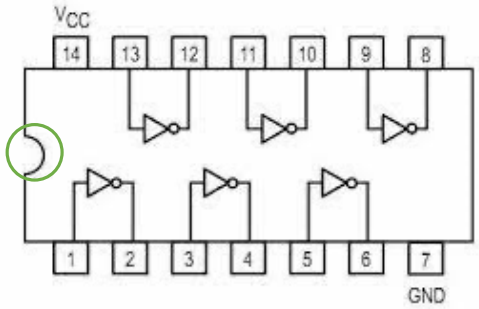
74LS00
Quad 2-input NAND



74LS02
Quad 2-input NOR



74LS04
Hex Inverter

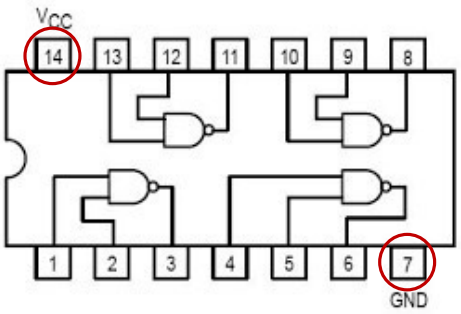


Marca orientadora

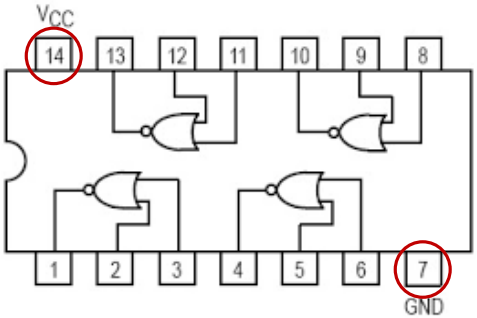
Circuitos integrados
Pinos

- Pinout

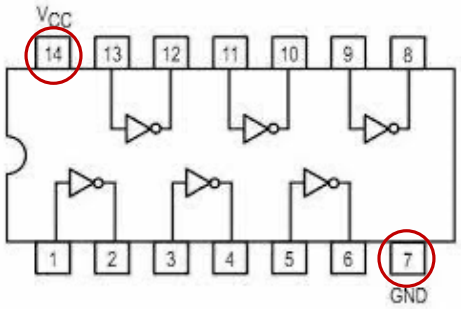
74LS00
Quad 2-input NAND



74LS02
Quad 2-input NOR



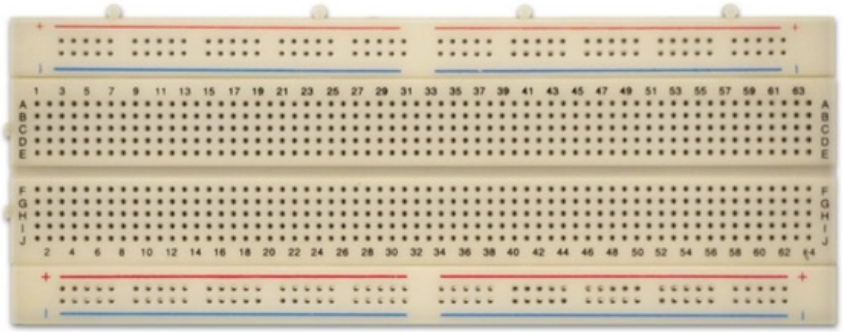
74LS04
Hex Inverter



Alimentações – Vcc e GND – não esquecer!

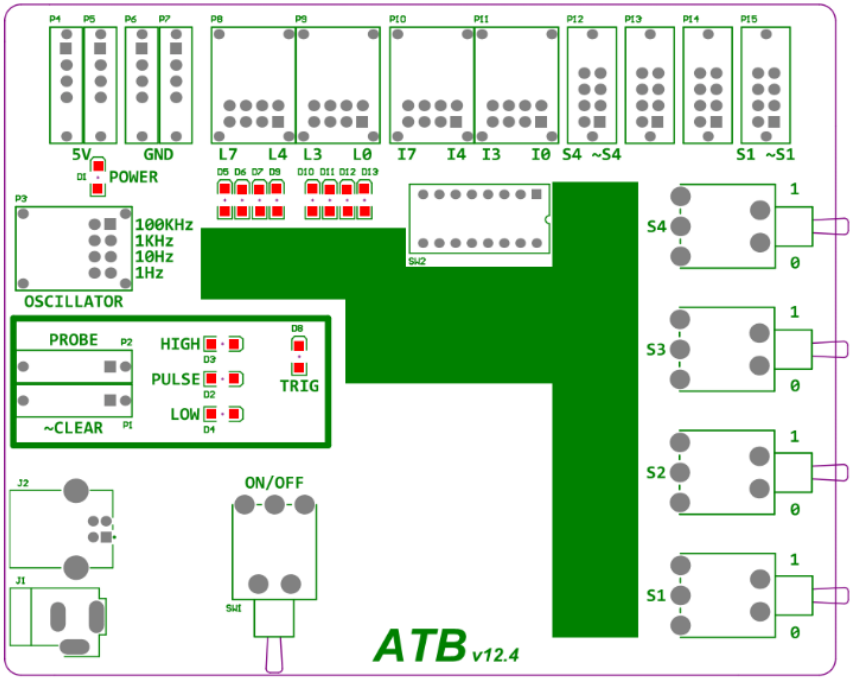
Equipamento no laboratório
Breadboard

- Dois grupos de *rails* longitudinais em cima e em baixo: tipicamente usados para alimentação (marcados pelos símbolos '+' e '-')
- *Rails* transversais: tipicamente usados para interligar os sinais aos pinos dos circuitos integrados
- Os circuitos integrados são colocados no centro da breadboard alinhados com o eixo longitudinal



Equipamento no laboratório ATB

- Advanced test board (ATB)
  - Disponibiliza a alimentação (5V e GND)
  - Disponibiliza comutadores (S1 – S4) para definição do valor lógica de cada uma das variáveis de entrada da função (A, B, C e D)
  - Disponibiliza LEDs (L0 – L7) para visualização de valores lógicos (usados para a saída)
  - Disponibiliza uma ponta de prova (Probe) que permite pesquisar valores lógicos em diversos pontos do circuito



Laboratório A
Fases de implementação

- A função a implementar deve estar representada através de uma expressão algébrica em que apenas seja usado um dos seguintes conjuntos de operadores: {NOT, NAND} ou {NOT, NOR}
- Escolher a referência dos circuitos que vão ser usados para implementar a função e determinar a quantidade necessária:
  - NOT – 74LS04
  - NAND – 74LS00
  - NOR – 74LS02
- Desenhar o esquema elétrico das ligações entre os pinos dos circuitos
- Usar a breadboard para colocar os circuitos integrados e efetuar as ligações
- Usar o ATB para fornecer a alimentação, definir os valores das entradas e visualizar a saída

Laboratório A
Fases de implementação (exemplo)

- Função a implementar: $F = AB + AC + BC$
  - NANDs e NOTs: $F = \overline{\overline{\overline{A \cdot B} \cdot \overline{A \cdot C}} \cdot \overline{B \cdot C}}$
  - NORs e NOTs: $F = \overline{\overline{\overline{A} + \overline{B}} + \overline{\overline{A} + \overline{C}} + \overline{\overline{B} + \overline{C}}}$

Laboratório A
Fases de implementação (exemplo)

- Função a implementar: $F = AB + AC + BC$

  - NANDs e NOTs: $F = \overline{\overline{\overline{\overline{A \cdot B} \cdot \overline{A \cdot C}} \cdot \overline{B \cdot C}}}$ 6 NANDs ou 5 NANDs e 1 NOT

  - NORs e NOTs: $F = \overline{\overline{\overline{\overline{\overline{A} + \overline{B}} + \overline{\overline{A} + \overline{C}}} + \overline{\overline{B} + \overline{C}}}}}$ 10 NORs ou 5 NORs e 5 NOTs

Laboratório A
Fases de implementação (exemplo)

- Função a implementar: $F = AB + AC + BC$

  - NANDs e NOTs: $F = \overline{\overline{\overline{\overline{A \cdot B} \cdot \overline{A \cdot C}} \cdot \overline{B \cdot C}}}$

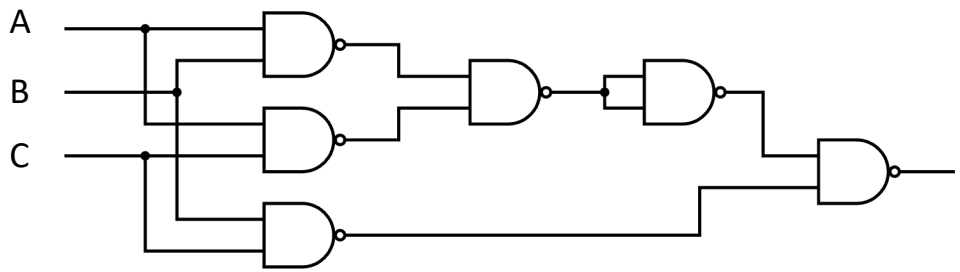
    6 NANDs ou 5 NANDs e 1 NOT
    2x74LS00 ou 2x74LS00 e 1x74LS04

  - NORs e NOTs: $F = \overline{\overline{\overline{\overline{\overline{A} + \overline{B}} + \overline{\overline{A} + \overline{C}}} + \overline{\overline{B} + \overline{C}}}}$

    10 NORs ou 5 NORs e 5 NOTs
    3x74LS02 ou 2x74LS02 e 1x74LS04

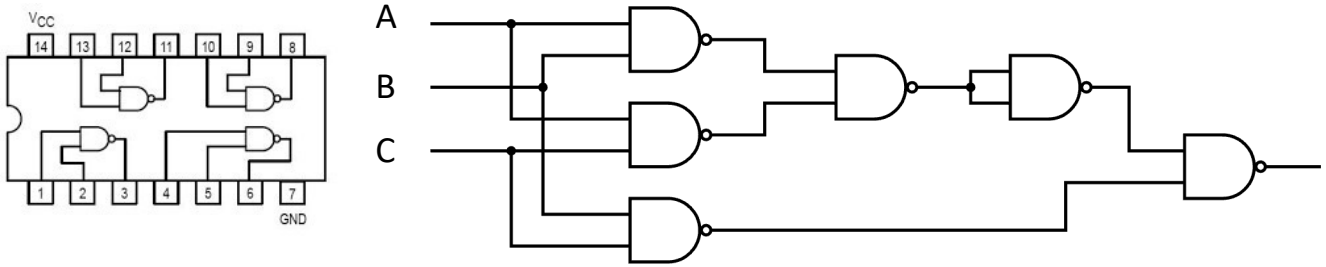
Laboratório A
Fases de implementação (exemplo)

- Função a implementar: $F = \overline{\overline{\overline{A \cdot B} \cdot \overline{A \cdot C} \cdot \overline{B \cdot C}}}$
- Logigrama:



Laboratório A
Fases de implementação (exemplo)

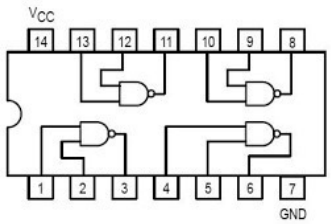
- Função a implementar: $F = \overline{\overline{\overline{A \cdot B} \cdot \overline{A \cdot C} \cdot \overline{B \cdot C}}}$
- Logigrama:
  - Adicionar identificadores das ligações nos pinos dos circuitos integrados (pode ser usada a ferramenta EasyEDA)

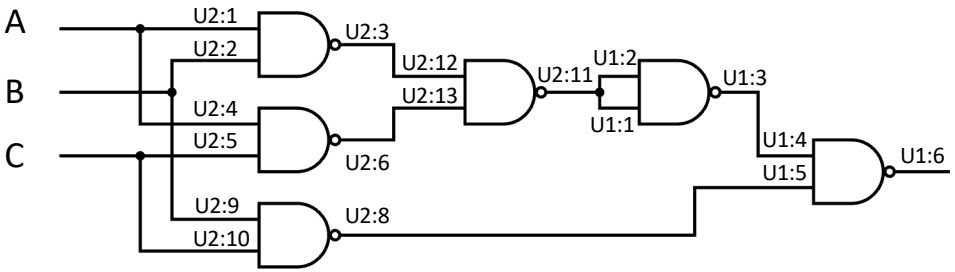


U1 – 74LS00
U2 – 74LS00

Laboratório A
Fases de implementação (exemplo)

- Função a implementar: $F = \overline{\overline{\overline{A \cdot B} \cdot \overline{A \cdot C} \cdot \overline{B \cdot C}}}$
- Logigrama:
  - Adicionar identificadores das ligações nos pinos dos circuitos integrados (pode ser usada a ferramenta EasyEDA)

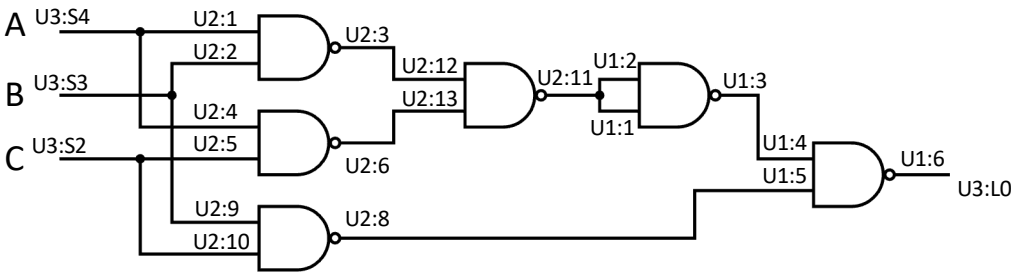




U1 – 74LS00
U2 – 74LS00

Laboratório A
Fases de implementação (exemplo)

- Função a implementar: $F = \overline{\overline{\overline{\overline{A \cdot B} \cdot \overline{A \cdot C}} \cdot \overline{B \cdot C}}}$
- Logigrama:
  - Adicionar identificadores das ligações nos pinos dos circuitos integrados (pode ser usada a ferramenta EasyEDA)



U1 – 74LS00
U2 – 74LS00
U3 – ATB

Laboratório A
Fases de implementação

- Efetuar as ligações na breadboard
  - Não esquecer alimentação dos circuitos integrados (Vcc e GND) –> ligar aos rails longitudinais existentes na breadboard (+ e -)

- Efetuar ligações entre ATB e breadboard
  - Garantir que a ATB está desligada
  - Ligar as entradas do circuito aos comutadores (S1 a S4)
  - Ligar a saída do circuito a um LED (e.g. L0)
  - Ligar alimentação do ATB aos rails longitudinais da breadboard
  - Depois de validar as ligações e de todos os fios estarem ligados (ambas as pontas), ligar a ATB

Laboratório A
Fases de implementação

- Função a implementar: $F = A \cdot \overline{B} \cdot \overline{C} + A \cdot B \cdot \overline{C} + \overline{A} \cdot B$
  - NANDs e NOTs: $F = \overline{\overline{A \cdot \overline{\overline{C}} \cdot \overline{\overline{A} \cdot B}}}$
  - NORs e NOTs: $F = \overline{\overline{\overline{\overline{A} + C} + \overline{A + \overline{\overline{B}}}}}$

Laboratório A
Fases de implementação

- Função a implementar: $F = A \cdot \overline{B} \cdot \overline{C} + A \cdot B \cdot \overline{C} + \overline{A} \cdot B$

  - NANDs e NOTs: $F = \overline{\overline{A \cdot \overline{\overline{C}} \cdot \overline{\overline{A}} \cdot B}}$

    5 NANDs ou 3 NANDs e 2 NOTs

  - NORs e NOTs: $F = \overline{\overline{\overline{\overline{A} + C} + \overline{A + \overline{B}}}}$

    6 NORs ou 3 NORs e 3 NOTs

Laboratório A
Fases de implementação

- Função a implementar: $F = A \cdot \overline{B} \cdot \overline{C} + A \cdot B \cdot \overline{C} + \overline{A} \cdot B$

  - NANDs e NOTs: $F = \overline{\overline{A \cdot \overline{\overline{C}} \cdot \overline{\overline{A} \cdot B}}}$

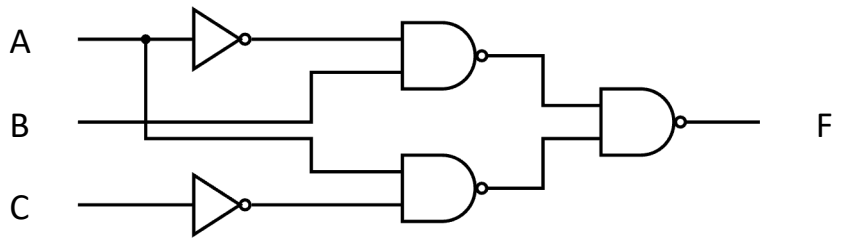
    5 NANDs ou 3 NANDs e 2 NOTs
    2x74LS00 ou 1x74LS00 e 1x74LS04

  - NORs e NOTs: $F = \overline{\overline{\overline{\overline{A} + C} + \overline{A + \overline{B}}}}$

    6 NORs ou 3 NORs e 3 NOTs
    2x74LS02 ou 1x74LS00 e 1x74LS02

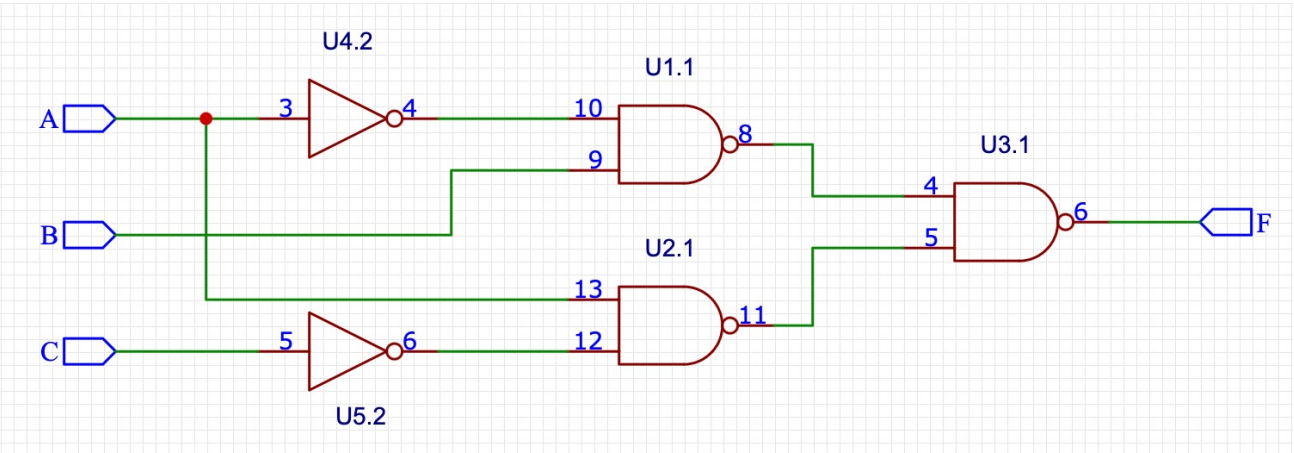
Laboratório A
Fases de implementação

- Função a implementar (NANDs e NOTs): $F = \overline{\overline{A \cdot \overline{C}} \cdot \overline{\overline{A} \cdot B}}$
- Logigrama:



Laboratório A
Fases de implementação

- Função a implementar (NANDs e NOTs): $F = \overline{\overline{A \cdot \overline{C}} \cdot \overline{\overline{A} \cdot B}}$
- Logigrama com anotação dos pinos (EasyEDA):



ISEL
DEETC
Departamento de
Engenharia Electrónica e
de Telecomunicações e
de Computadores

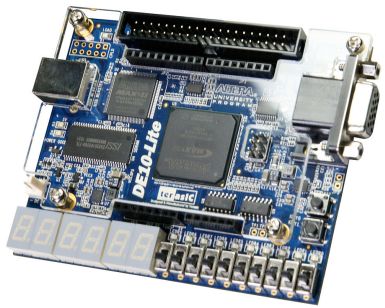
Laboratório A – Parte II

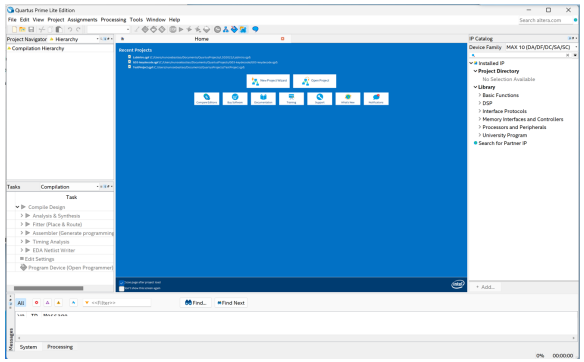
Funções Lógicas – Tecnologia *FPGA*

ISEL
DEETC
Departamento de
Engenharia Electrónica e
de Telecomunicações e
de Computadores

Implementação
Lógica programável - FPGA

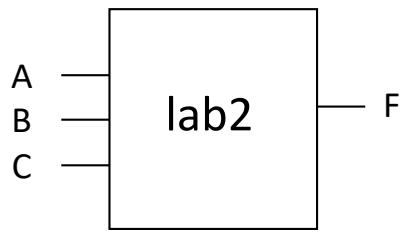
- Placa de desenvolvimento a ser usada no laboratório:
  - MAX DE10-Lite
    - Contém uma FPGA Altera MAX 10
- Ferramentas EDA
  - Quartus Prime Lite Edition
- Linguagem de descrição de hardware
  - VHSIC Hardware Description Language (VHDL)
    - VHSIC – Very High Speed Integrated Circuit





Descrição do circuito em VHDL (Laboratório 1 – parte II)

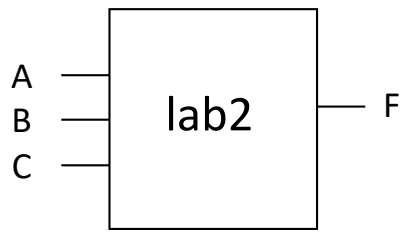
- Estrutura de um ficheiro VHDL
  - *Entity* (entidade)
    - Utilizada para descrever o interface (entradas, saídas e configurações) do circuito
    - Corresponde à vista do exterior do circuito
  - *Architecture* (arquitetura)



$$F(A, B, C) = A\overline{C} + \overline{A}B$$

Descrição do circuito em VHDL (Laboratório 1 – parte II)

- Estrutura de um ficheiro VHDL
  - *Entity* (entidade)
    - Utilizada para descrever o interface (entradas, saídas e configurações) do circuito
    - Corresponde à vista do exterior do circuito

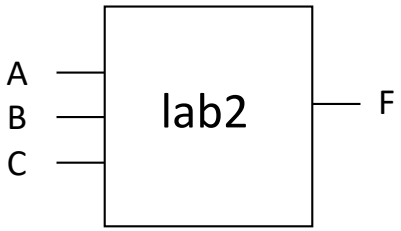


$$F(A, B, C) = A\overline{C} + \overline{A}B$$

  - *Architecture* (arquitetura)
    - Utilizada para descrever a funcionalidade da entidade
    - Pode ser descrita usando duas formas:
      - *Structural* (estrutural) – descrição usando outros componentes
      - *Behavioral* (comportamental) – descrição do comportamento

Descrição do circuito em VHDL (Laboratório 1 – parte II)

- Estrutura de um ficheiro VHDL

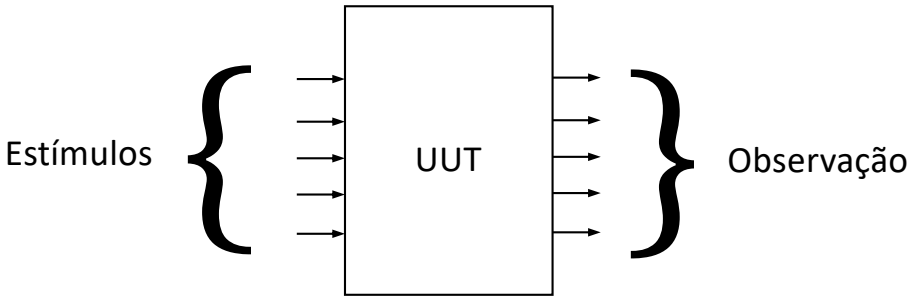


$$F(A, B, C) = A\overline{C} + \overline{A}B$$

```
library ieee;
use ieee.std_logic_1164.all;

entity lab2 is
   port (
      A : in std_logic;
      B : in std_logic;
      C : in std_logic;
      F : out std_logic
   );
end lab2;

architecture behavioral of lab2 is

begin

   F <= (A and not C) or (not A and B);

end behavioral;
```

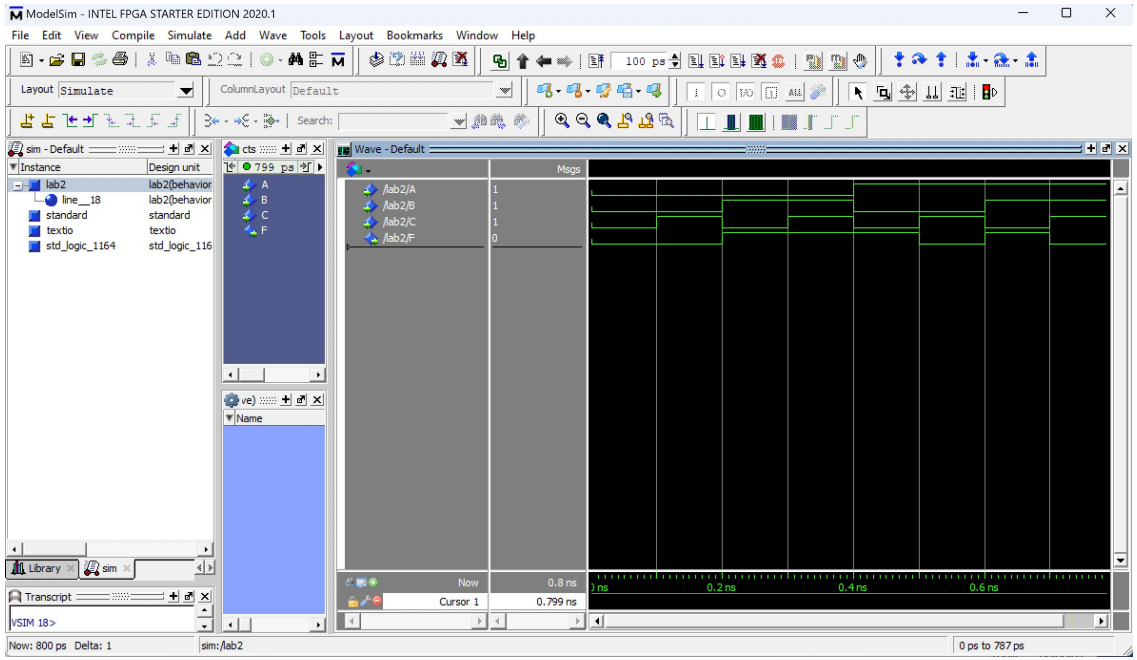
Teste / validação

- UUT – *Unit under test*



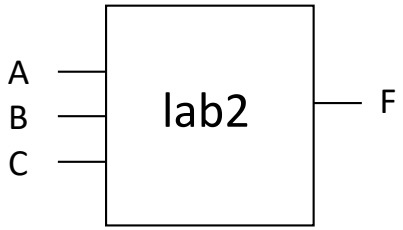
Teste através de simulação

- Simulação com estímulos definidos manualmente



Validação na placa MAX DE10-Lite

- Atribuição de pinos
  - É necessário mapear as entradas e saídas do circuito nos pinos disponíveis da FPGA.
  - No caso da placa de desenvolvimento utilizada, cada pino da FPGA está interligado a um periférico específico (e.g. LED, switch)
  - É necessário consultar o datasheet da placa para se identificar qual o pino a que se quer ligar cada entrada e saída do circuito
    - A, B, C – interruptores
    - F - LED

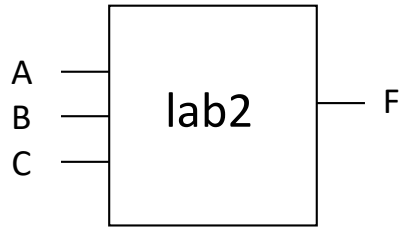


Validação na placa MAX DE10-Lite

- Atribuição de pinos (inputs)

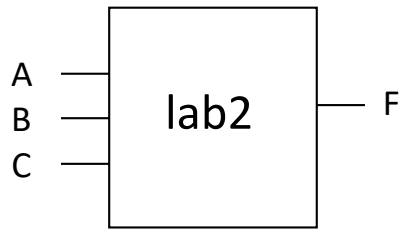
Validação na placa MAX DE10-Lite

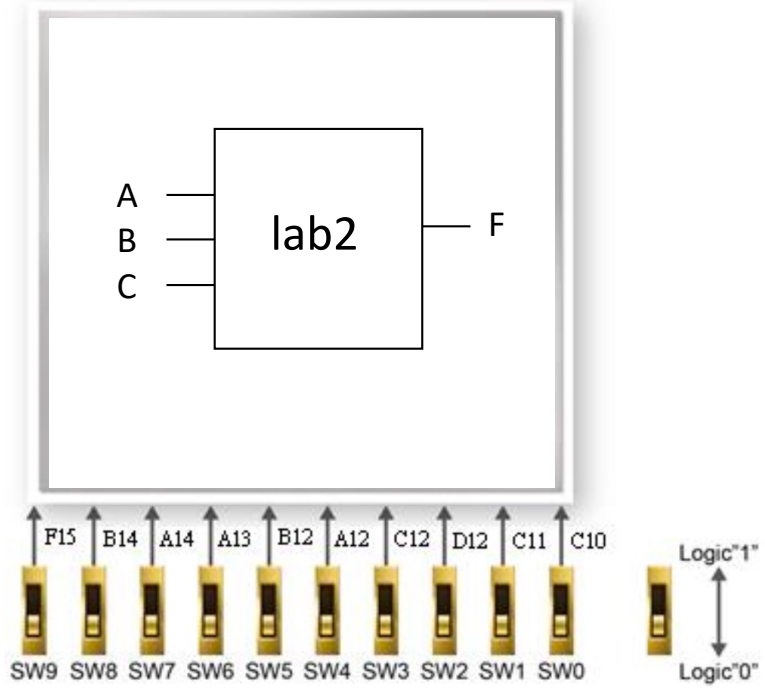
- Atribuição de pinos (inputs)



Validação na placa MAX DE10-Lite

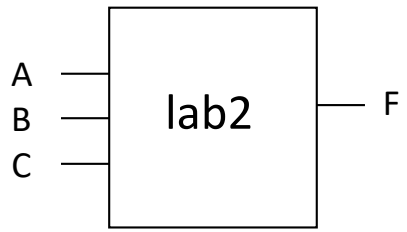
- Atribuição de pinos (inputs)

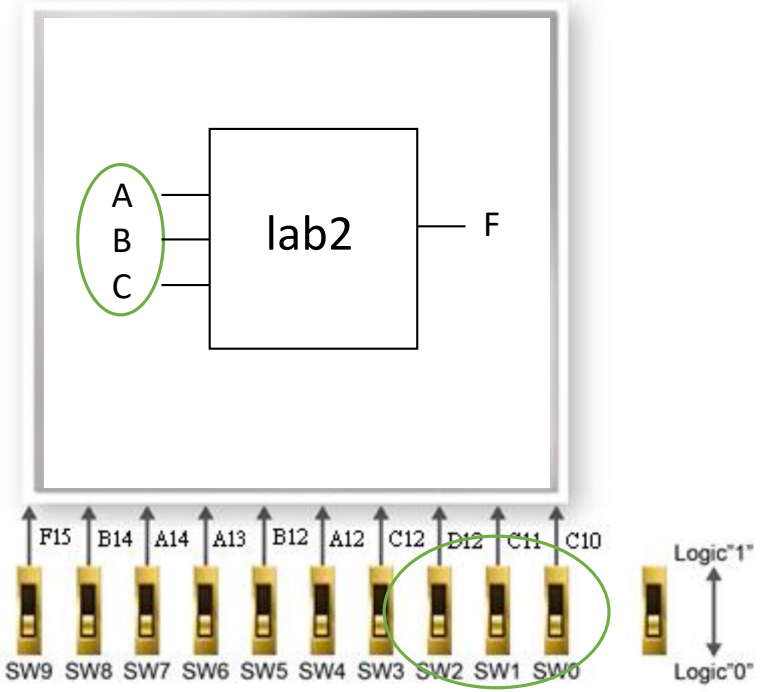




Validação na placa MAX DE10-Lite

- Atribuição de pinos (inputs)

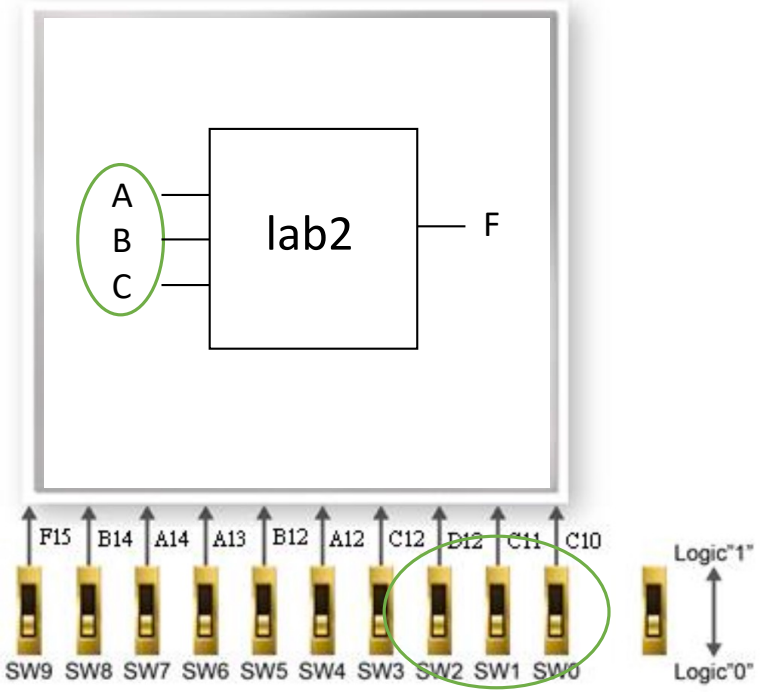




Validação na placa MAX DE10-Lite

- Atribuição de pinos (inputs)

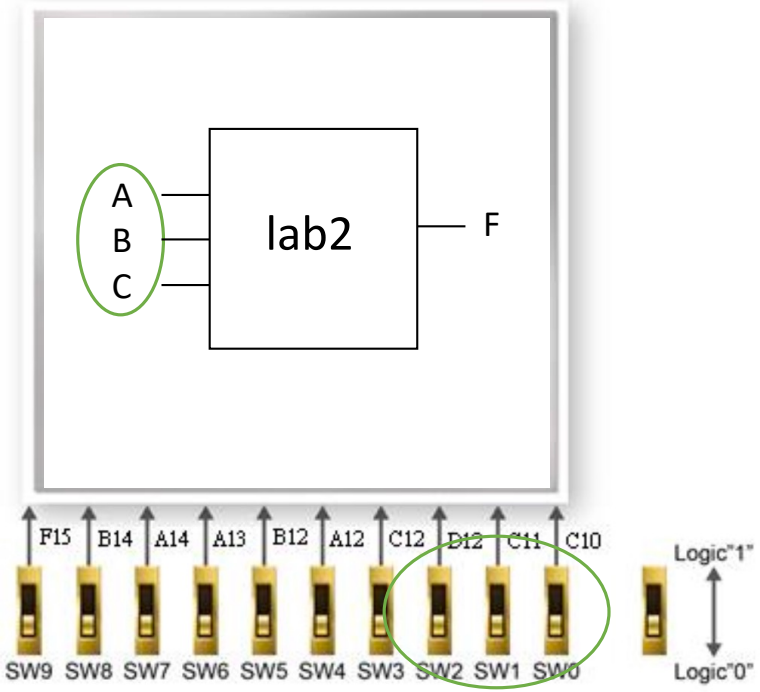
Signal Name	FPGA Pin No.	Description	I/O Standard
SW0	PIN_C10	Slide Switch[0]	3.3-V LVTTL
SW1	PIN_C11	Slide Switch[1]	3.3-V LVTTL
SW2	PIN_D12	Slide Switch[2]	3.3-V LVTTL
SW3	PIN_C12	Slide Switch[3]	3.3-V LVTTL
SW4	PIN_A12	Slide Switch[4]	3.3-V LVTTL
SW5	PIN_B12	Slide Switch[5]	3.3-V LVTTL
SW6	PIN_A13	Slide Switch[6]	3.3-V LVTTL
SW7	PIN_A14	Slide Switch[7]	3.3-V LVTTL
SW8	PIN_B14	Slide Switch[8]	3.3-V LVTTL
SW9	PIN_F15	Slide Switch[9]	3.3-V LVTTL



Validação na placa MAX DE10-Lite

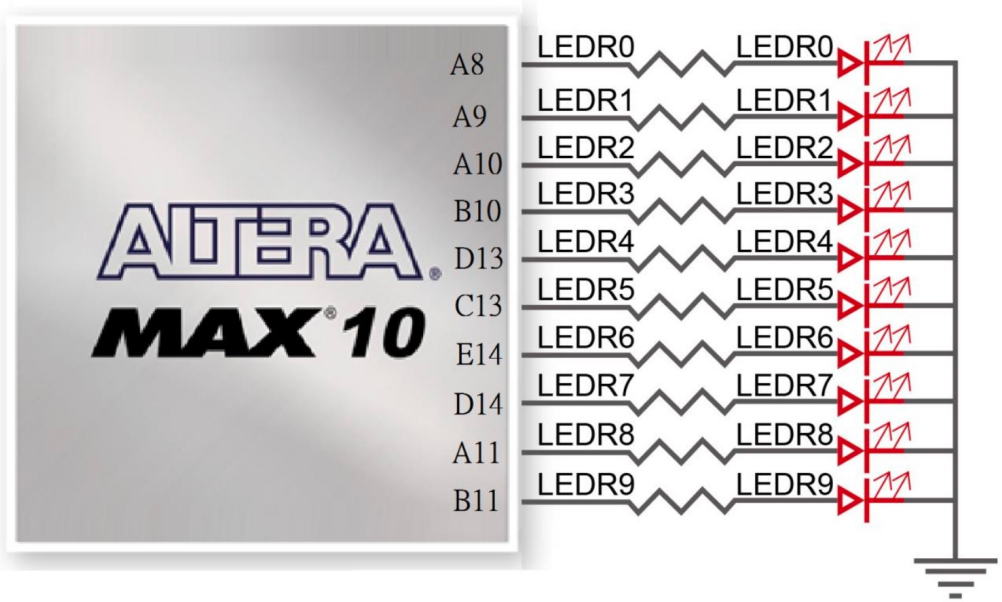
- Atribuição de pinos (inputs)

Signal Name	FPGA Pin No.	Description	I/O Standard
SW0	PIN_C10	Slide Switch[0]	3.3-V LVTTL
SW1	PIN_C11	Slide Switch[1]	3.3-V LVTTL
SW2	PIN_D12	Slide Switch[2]	3.3-V LVTTL
SW3	PIN_C12	Slide Switch[3]	3.3-V LVTTL
SW4	PIN_A12	Slide Switch[4]	3.3-V LVTTL
SW5	PIN_B12	Slide Switch[5]	3.3-V LVTTL
SW6	PIN_A13	Slide Switch[6]	3.3-V LVTTL
SW7	PIN_A14	Slide Switch[7]	3.3-V LVTTL
SW8	PIN_B14	Slide Switch[8]	3.3-V LVTTL
SW9	PIN_F15	Slide Switch[9]	3.3-V LVTTL



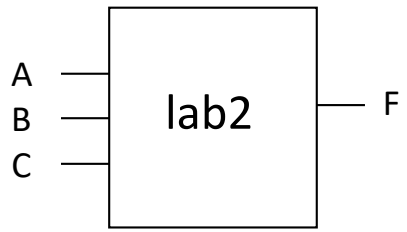
Validação na placa MAX DE10-Lite

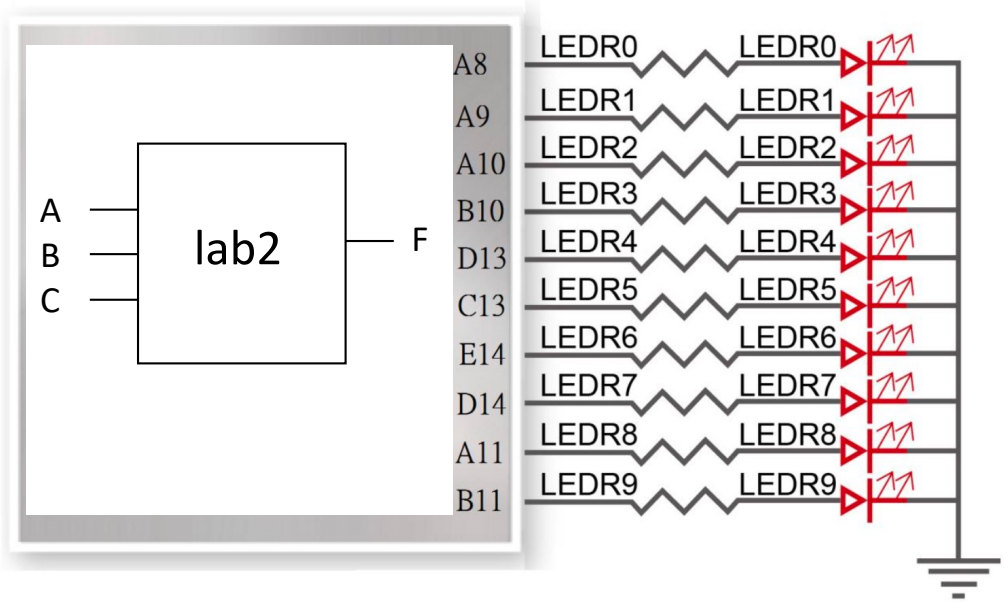
- Atribuição de pinos (outputs)



Validação na placa MAX DE10-Lite

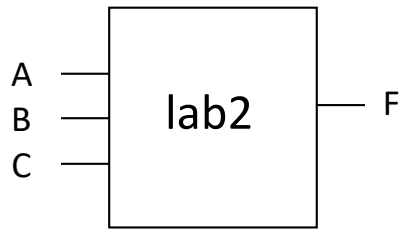
- Atribuição de pinos (outputs)

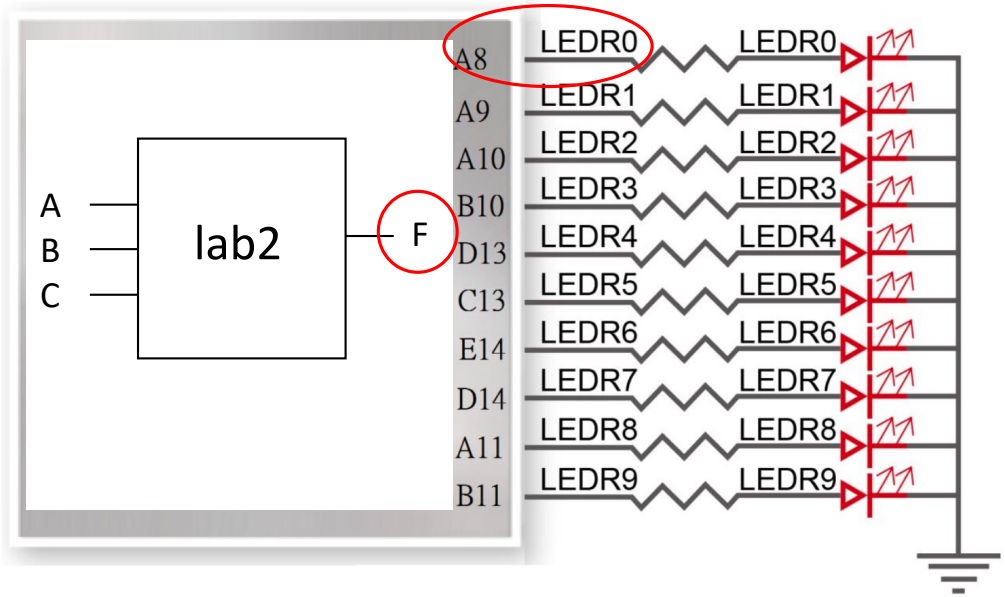




Validação na placa MAX DE10-Lite

- Atribuição de pinos (outputs)

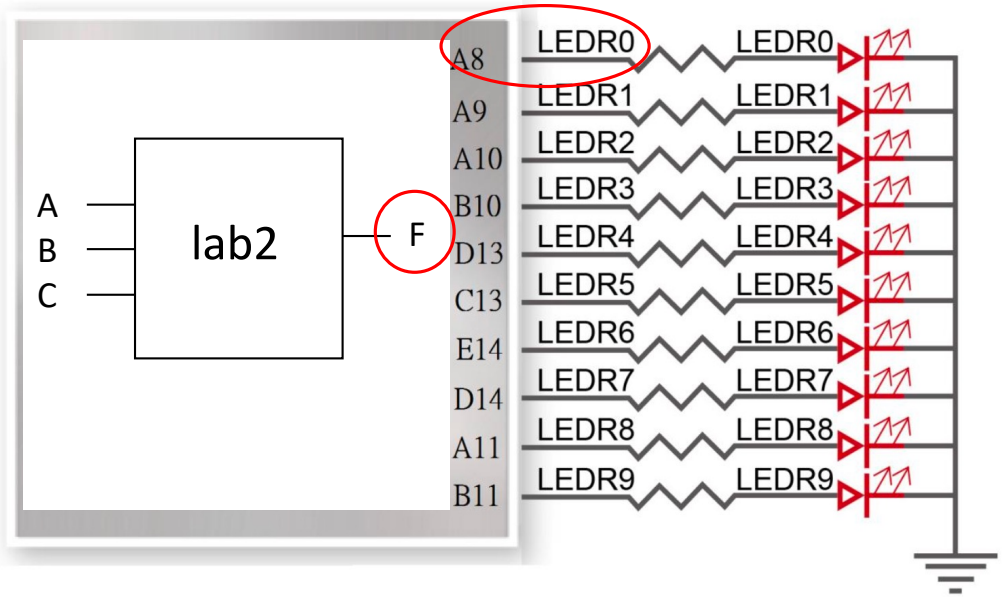




Validação na placa MAX DE10-Lite

- Atribuição de pinos (outputs)

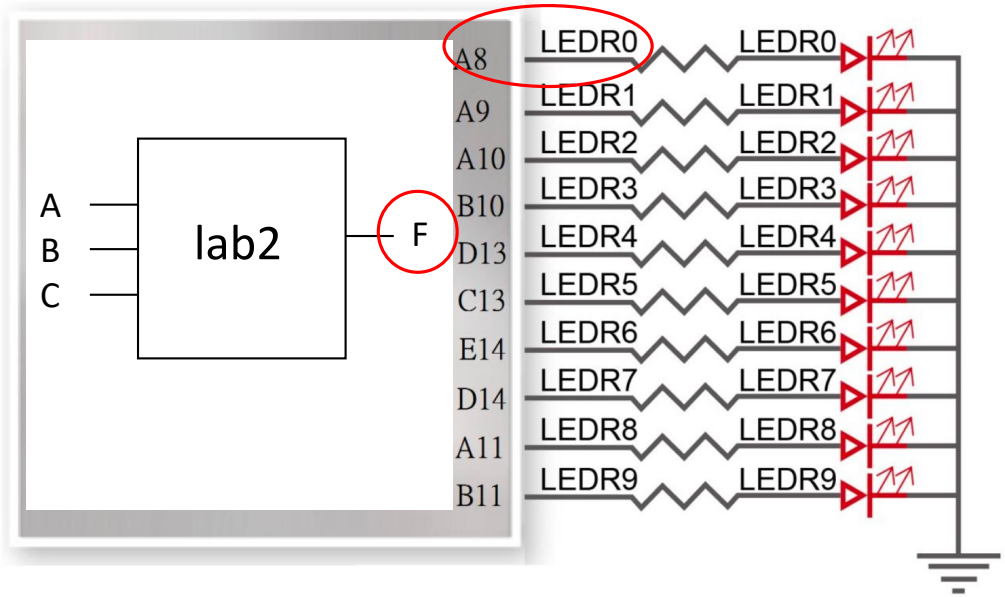
Signal Name	FPGA Pin No.	Description	I/O Standard
LEDR0	PIN_A8	LED [0]	3.3-V LVTTL
LEDR1	PIN_A9	LED [1]	3.3-V LVTTL
LEDR2	PIN_A10	LED [2]	3.3-V LVTTL
LEDR3	PIN_B10	LED [3]	3.3-V LVTTL
LEDR4	PIN_D13	LED [4]	3.3-V LVTTL
LEDR5	PIN_C13	LED [5]	3.3-V LVTTL
LEDR6	PIN_E14	LED [6]	3.3-V LVTTL
LEDR7	PIN_D14	LED [7]	3.3-V LVTTL
LEDR8	PIN_A11	LED [8]	3.3-V LVTTL
LEDR9	PIN_B11	LED [9]	3.3-V LVTTL



Validação na placa MAX DE10-Lite

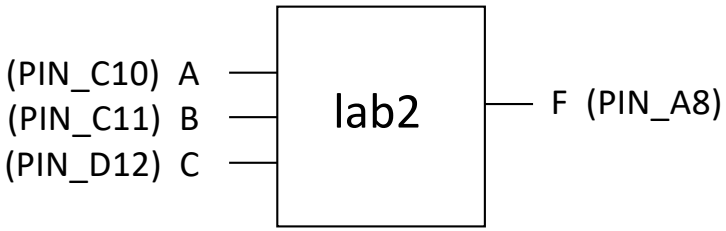
- Atribuição de pinos (outputs)

Signal Name	FPGA Pin No.	Description	I/O Standard
LEDR0	PIN_A8	LED [0]	3.3-V LVTTL
LEDR1	PIN_A9	LED [1]	3.3-V LVTTL
LEDR2	PIN_A10	LED [2]	3.3-V LVTTL
LEDR3	PIN_B10	LED [3]	3.3-V LVTTL
LEDR4	PIN_D13	LED [4]	3.3-V LVTTL
LEDR5	PIN_C13	LED [5]	3.3-V LVTTL
LEDR6	PIN_E14	LED [6]	3.3-V LVTTL
LEDR7	PIN_D14	LED [7]	3.3-V LVTTL
LEDR8	PIN_A11	LED [8]	3.3-V LVTTL
LEDR9	PIN_B11	LED [9]	3.3-V LVTTL



Validação na placa MAX DE10-Lite

- Atribuição de pinos




```
set_global_assignment -name BOARD "MAX 10 DE10 - Lite"
set_global_assignment -name DEVICE 10M50DAF484C6GES
set_global_assignment -name FAMILY "MAX 10"

set_location_assignement PIN_A8 -to F
set_location_assignement PIN_C10 -to A
set_location_assignement PIN_C11 -to B
set_location_assignement PIN_D12 -to C
```

Validação na placa MAX DE10-Lite

- Programação
  - Após compilar a descrição, é gerado um ficheiro de programação da FPGA
  - Este ficheiro deve ser enviado para a FPGA usando o programador

Temas abordados

Sumário

- Exercícios de simplificação de funções com mapa de Karnaugh
- VHDL - descrição estrutural
- Introdução ao laboratório A

Próxima aula

- Circuitos combinatórios